MLP và backpropagation

Lỗi đầu ra quay về lớp đầu như thế nào?

Hai công tắc, một đèn

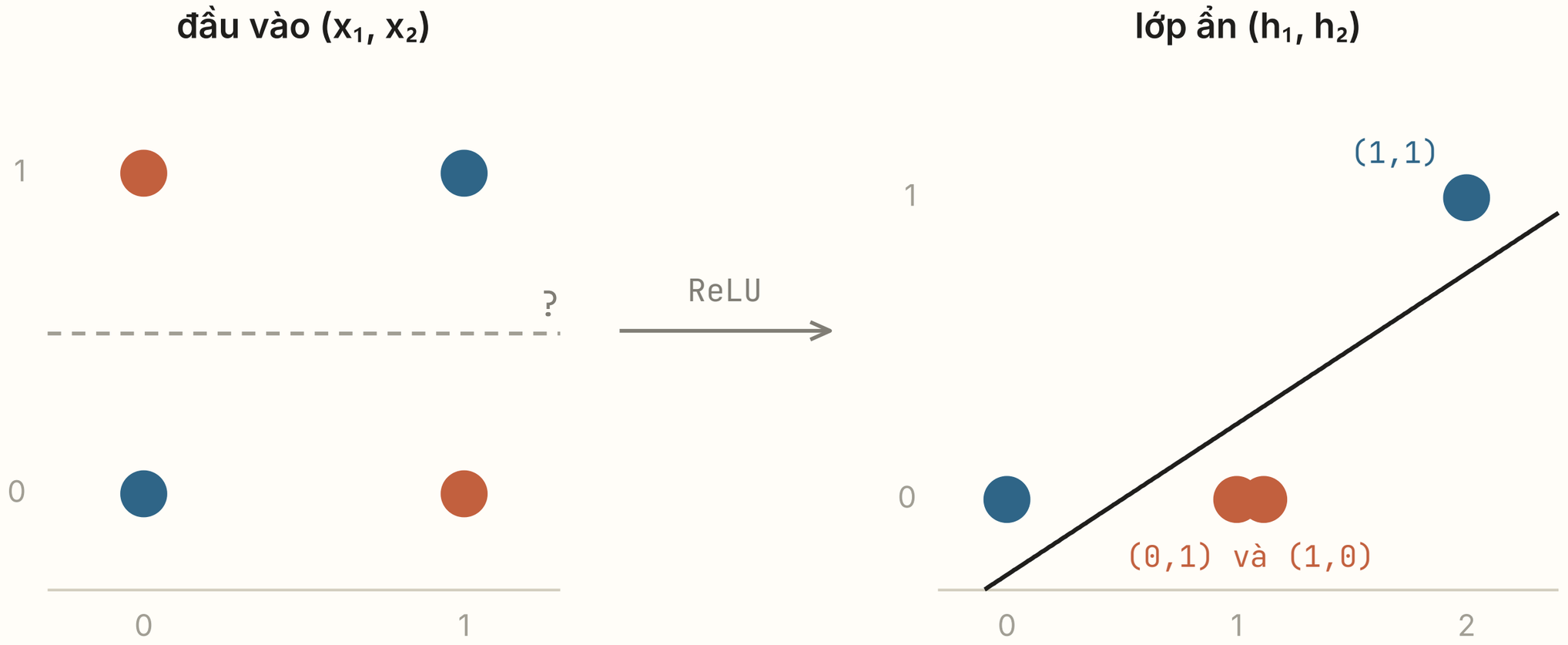
Đèn sáng khi đúng một công tắc bật.

XOR: 00→0, 01→1, 10→1, 11→0.

Một đường thẳng có tách được hai lớp?

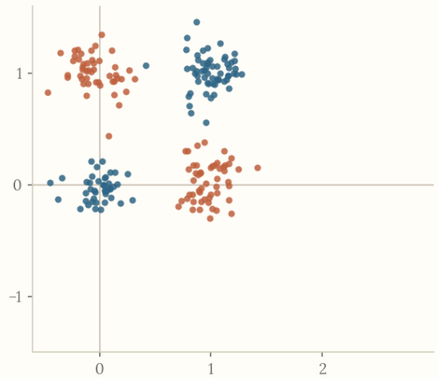
Thêm epoch chưa chắc sửa được hạn chế biểu diễn.

Nhìn bốn góc

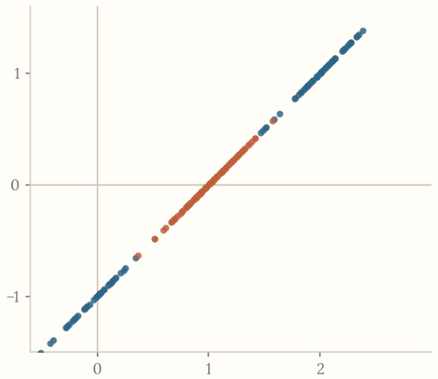


Trong mặt phẳng đầu vào, không đường thẳng nào tách hai cặp góc XOR; qua lớp ẩn ReLU, hai góc nhãn 1 dồn về (1, 0) và một đường thẳng tách được.

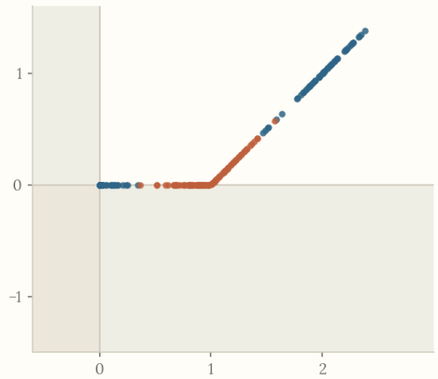
Lớp ẩn gập mặt phẳng



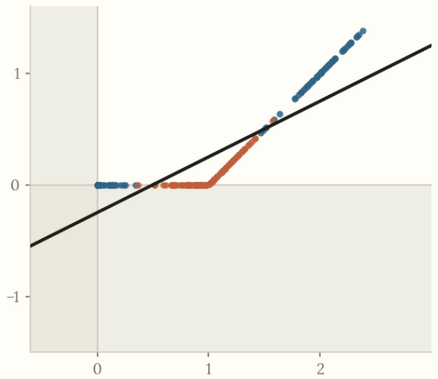
đầu vào (x_1, x_2): không đường thẳng nào tách hai cặp góc



z = xW + b với W toàn 1, b = (0, −1): $z_2 = z_1 - 1$, mọi điểm dồn lên một đường thẳng



ReLU: tọa độ âm bị ép về 0, nửa dưới gập lên trục ngang



$h_1 - 2h_2 = 0.5$ tách hai lớp: đúng 98% trong 200 điểm

Lớp ẩn đặt tay của bài biến 200 điểm XOR: phép tuyến tính dồn mọi điểm lên một đường thẳng, ReLU gập phần âm về trục, rồi một đường thẳng tách được hai lớp.

Tạo hai đặc trưng trung gian

$$h_1 = \mathrm{ReLU}(x_1 + x_2).$$

$$h_2 = \mathrm{ReLU}(x_1 + x_2 - 1).$$

$$\hat{y} = h_1 - 2h_2.$$

Trọng số đặt tay để kiểm khả năng biểu diễn.

Tổng 0, 1, 2

Tổng input	h_1	h_2	Output
0	0	0	0
1	1	0	1
2	2	1	0

Kết quả code:

KẾT QUẢ

```
[0. 1. 1. 0.]
```

Activation tạo phi tuyến.

HỎI NHANH

Giữ nguyên hai đặc trưng h_1, h_2 của mạng XOR đặt tay, chỉ đổi đầu ra thành $\hat{y} = h_1 - h_2$. Với bốn đầu vào $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$, mạng cho ra hàm nào?

A OR: $0, 1, 1, 1$

B AND: $0, 0, 0, 1$

C Vẫn là XOR: $0, 1, 1, 0$

D NAND: $1, 1, 1, 0$

HỎI NHANH

Mạng $2 \to 100 \to 1$ gồm hai lớp linear nối thẳng, không có activation ở giữa. Mạng này biểu diễn được những hàm nào?

A Mọi hàm liên tục, nhờ 100 neuron ẩn

B Đúng những hàm mà một lớp linear biểu diễn được

C Thêm được XOR, nhờ lớp ẩn tạo ra đặc trưng mới

D Nhiều hơn một lớp linear, nhưng chưa tới XOR

01

Forward và shape

Mẫu đặt ở hàng

B: số mẫu; d: feature vào; h: neuron ẩn; c: số đầu ra.

$$Z_1 = XW_1 + b_1.$$

$$H = \mathrm{ReLU}(Z_1).$$

$$P = HW_2 + b_2.$$

Tensor	Shape
X	`(B, d)`
W_1	`(d, h)`
H	`(B, h)`
W_2	`(h, c)`
P	`(B, c)`

Hai quy ước, một phép tính

NumPy trong bài: $Z = XW_1$, `W1` shape `(d,h)`.

`nn.Linear(d,h)`: $Z = X\,\mathrm{weight}^\top + b$, `weight` shape `(h,d)`.

Chép trọng số tự cài sang `Linear`: gán `W1.T`.

HỎI NHANH

Lớp đầu đã được cài bằng NumPy theo quy ước hàng, $Z = XW_1$, với `W1` có shape `(3, 3)`. Chép thẳng `W1` vào `weight` của `torch.nn.Linear(3, 3)` rồi chạy. Chuyện gì xảy ra?

A Báo lỗi shape ngay ở lần forward đầu tiên

B Ra đúng kết quả, vì PyTorch tự chuyển vị khi cần

C Chạy êm, nhưng tính ra $XW_1^\top$, sai mà không báo

D Chỉ bias bị sai, phần trọng số vẫn đúng

Đầu ra hồi quy một chiều

Với hồi quy một đầu ra và MSE trung bình trên B mẫu:

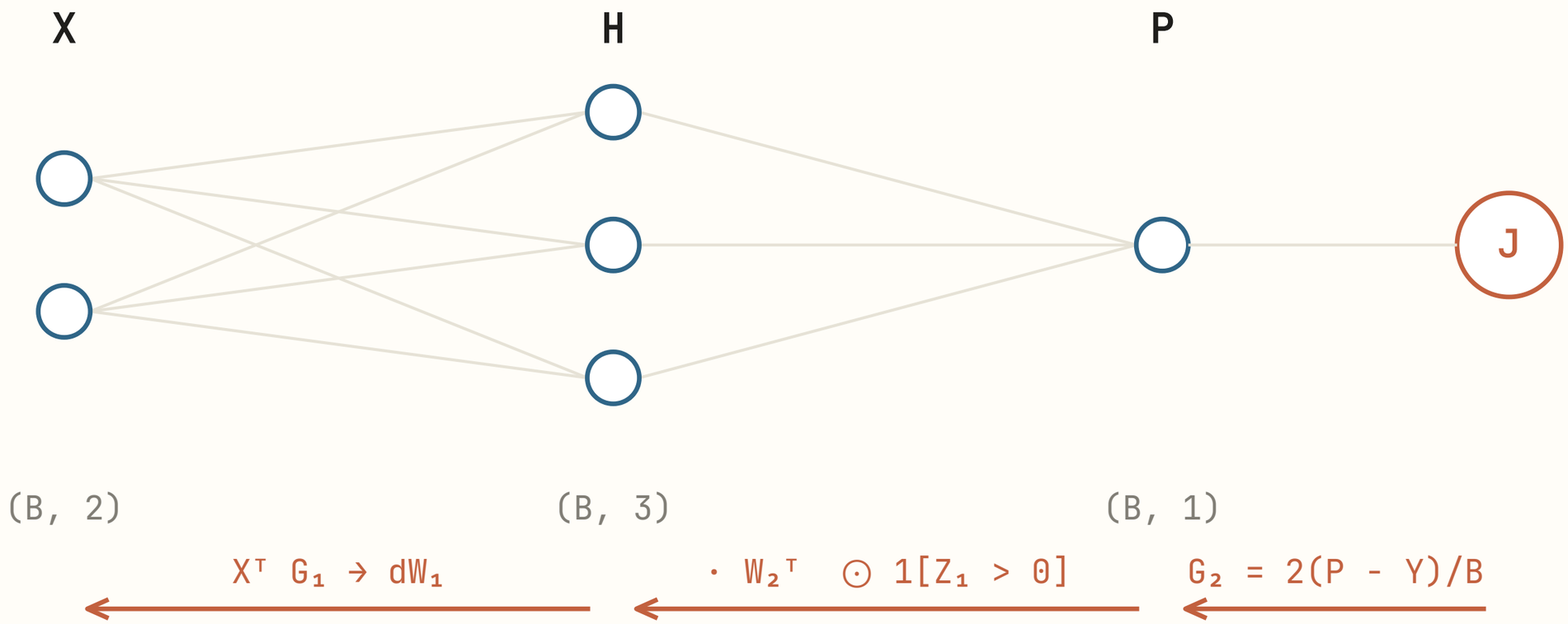
$$G_2 = \frac{2}{B}(P - Y).$$

$$\nabla_{W_2} J = H^\top G_2.$$

$$G_1 = (G_2 W_2^\top) \odot \mathbf{1}[Z_1 > 0].$$

Gradient phải có cùng shape tham số.

Gradient đi ngược mạng 2→3→1



Mạng 2 → 3 → 1: forward đi từ trái sang; gradient đi ngược từ J, nhân W_2^T để về lớp ẩn, qua cổng ReLU′ rồi mới tới W_1.

HỎI NHANH

`H` có shape `(5, 4)`, `G2` có shape `(5, 1)`. Gradient theo `W2` có shape gì?

A `(5, 5)`

B `(5, 4)`

C `(1, 4)`

D `(4, 1)`

02

Tính và kiểm

Một mẫu tính tay

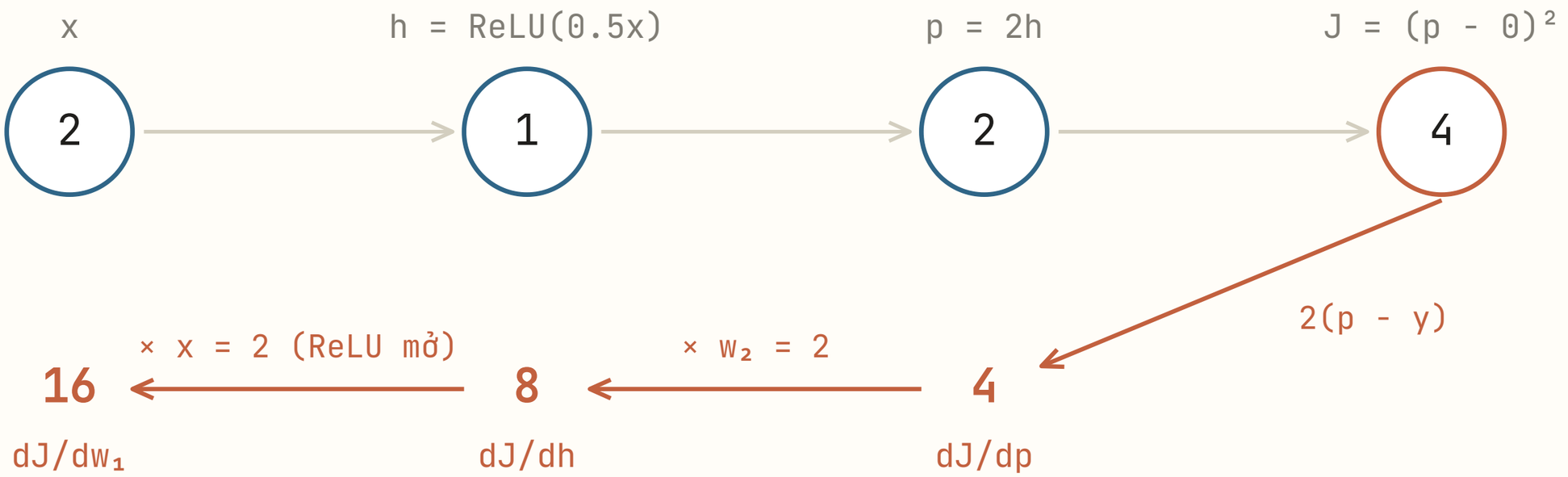
$x = 2$, $y = 0$, $w_1 = 0.5$, $b_1 = 0$, $w_2 = 2$, $b_2 = 0$.

$z = 1 \to h = 1 \to p = 2 \to J = (p - y)^2 = 4$.

- $\partial J / \partial p = 4$
- $\partial J / \partial w_2 = 4$
- $\partial J / \partial h = 8$
- $\partial J / \partial w_1 = \mathbf{16}$

Dùng trọng số cũ trong toàn bộ backward.

Forward và backward của một mẫu



Ví dụ tính tay x = 2, y = 0: forward 2 → 1 → 2 → 4; backward nhân dần 4 → 8 → 16 để ra $dJ/dw_1 = 16$.

HỎI NHANH

Mạng một mẫu có $h = \mathrm{ReLU}(w_1 x)$, $p = w_2 h$, $L = (p - y)^2$. Tại $x = 3$, $w_1 = 1$, $w_2 = 1$, $y = 1$, gradient $\partial L / \partial w_1$ bằng bao nhiêu?

A 6

B 3

C 12

D 4

NumPy: từng phép có tên

Đoạn trích từ hàm `backward` trong notebook: `params[2]` là W_2, `Y` là target cùng shape với `P`.

```
G2 = 2 * (P - Y) / len(X)
G1 = (G2 @ params[2].T) * (Z1 > 0)
```

Đầu ra của batch kiểm:

KẾT QUẢ

```
[0.56 0.1 ]
0.1018
```

Target hai mẫu là 1 và 0, nên loss $= \big((0.56-1)^2 + (0.1-0)^2\big)/2 = 0.1018.$

Mask của ReLU



mẫu 2, neuron 2 đóng:

gradient qua ô này bằng 0

hàng là mẫu, cột là neuron ẩn

Ở batch hai mẫu của bài, Z_1 chỉ có một ô âm; mask ($Z_1 > 0$) chặn gradient qua đúng ô đó, ba ô còn lại cho gradient đi qua.

HỎI NHANH

Hàm `backward` ở trên bị quên `* (Z1 > 0)`. Với Z_1 của batch này, những gradient nào bị sai?

A Cả bốn gradient, vì G_1 có mặt trong mọi công thức

B Chỉ phần W_1, b_1 nối vào neuron ẩn thứ hai

C Chỉ W_2 và b_2, vì chúng nằm sát ReLU nhất

D Không gradient nào, vì forward đã cắt H về 0

Bias nhận tổng đóng góp

```
G1.sum(axis=0)
G2.sum(axis=0)
```

Đã chia B trong G_2, không chia thêm lần nữa.

Gradient theo W_2:

KẾT QUẢ

```
[[-0.474]
 [-0.044]]
```

-0.474 là tổng đóng góp hai mẫu.

Ba cách đối chiếu

Backward NumPy · sai phân trung tâm · autograd

KẾT QUẢ

```
numeric match: True
autograd match: True
0.1018 0.065513
```

Hai cách độc lập khớp backward; một bước lr 0.05 giảm loss.

HỎI NHANH

Mạng nhỏ ở trên có 9 phần tử tham số. Kiểm bằng sai phân trung tâm thì phải tính `objective` bao nhiêu lần? Với một MLP có 10^6 tham số thì sao?

A 18 lần, và mạng 10^6 tham số cần $2 \cdot 10^6$ lần

B 9 lần, và mạng 10^6 tham số cần 10^6 lần

C 2 lần, số tham số không ảnh hưởng chi phí

D 4 lần, mỗi tensor một cặp, và mạng lớn vẫn chỉ 4 lần

HỎI NHANH

Gradient tính bằng NumPy trùng với autograd và với sai phân trên một batch nhỏ. Điều đó bảo đảm được gì?

A Mạng sẽ có điểm validation cao sau khi train

B Gradient descent sẽ tới được nghiệm toàn cục

C Mọi batch khác cũng sẽ cho loss giảm

D Backward tính đúng đạo hàm tại điểm đã kiểm

Dùng nn.Linear

Mạng 2→3→1 có 13 tham số tính cả bias.

Linear(2,3) lưu weight `(3,2)`.

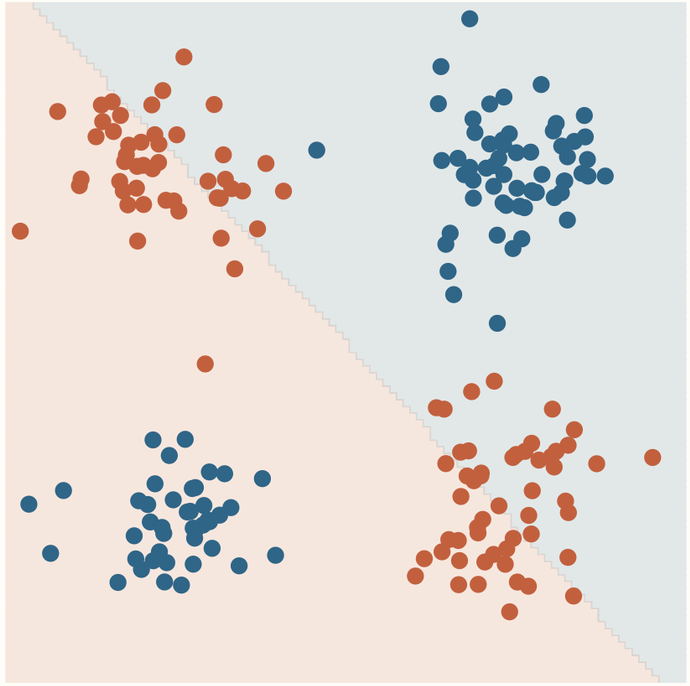
KẾT QUẢ

```
(4, 1)
13
```

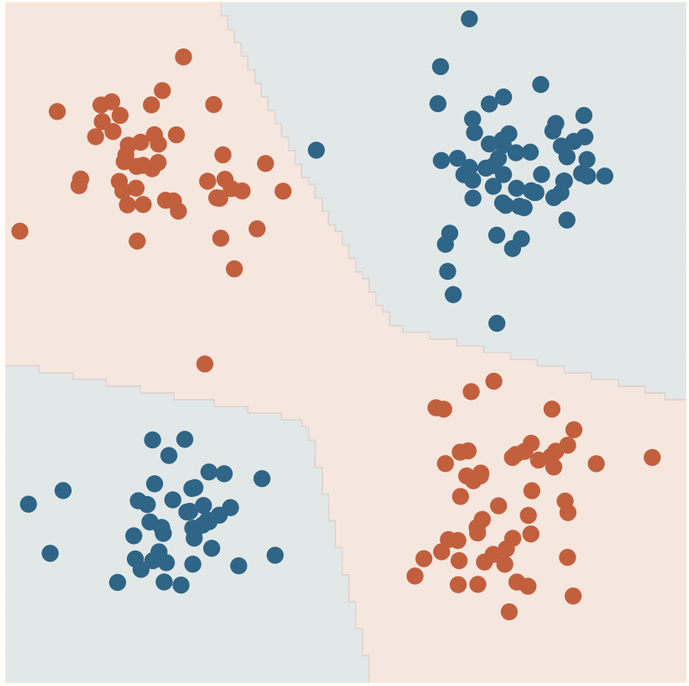
Chuyển NumPy weight `(2,3)` sang Linear cần chuyển vị.

Học XOR thật

linear (không lớp ẩn) · train 57%



MLP 2 → 8 → 1 · train 100%



200 điểm quanh bốn góc XOR, cùng 500 bước Adam: mô hình linear chỉ đúng 57%; thêm một lớp ẩn 8 neuron ReLU thì học được cả bốn vùng.

HỎI NHANH

Mạng hồi quy trả về `pred` shape `(4, 1)`, giá trị đúng bằng target `y = torch.tensor([1., 2., 3., 4.])` có shape `(4,)`. Thế nhưng `torch.nn.MSELoss()(pred, y)` lại cho 2.5 thay vì 0. Vì sao?

A `MSELoss` mặc định cộng các bình phương thay vì lấy trung bình

B Sai số làm tròn của `float32` cộng dồn qua bốn mẫu

C `pred - y` bị broadcast thành `(4, 4)`, so mọi cặp mẫu

D `y` phải đổi sang `float64` cho cùng kiểu với `pred`

Khi mạng không học

Kiểm shape · ReLU đóng · detach · zero_grad.

Tính đủ gradient rồi mới cập nhật.

Chi phí forward chính:

$$O(Bdh + Bhc).$$

Tăng width tăng cả capacity và tài nguyên.

Khi đi thi

- Học thuộc vài chục mẫu trước; không được thì lỗi ở code.
- Tự viết backward: kiểm sai phân, `float64`, mạng tí hon.
- Đếm tham số, bộ nhớ; so với baseline linear cùng seed.
- Đặt trọng số tay: neuron ẩn là nửa mặt phẳng, ghép AND/OR.
- Chép sang `nn.Linear`: chuyển vị rồi so output.

03

Luyện đề

LUYỆN ĐỀ 1

Đề yêu cầu dựng một MLP chỉ gồm lớp linear và hàm ngưỡng $\Theta(z) = 1$ nếu $z \geq 0$, ngược lại bằng 0. Trọng số đặt tay, không huấn luyện. Đầu ra phải bằng 1 khi điểm (x, y) nằm trong tam giác có đỉnh $(0, 0)$, $(3, 1)$, $(1, 5)$, tính cả cạnh. Lớp ẩn có $h_1 = \Theta(3y - x)$, $h_2 = \Theta(7 - 2x - y)$, $h_3 = \Theta(5x - y)$. Chọn neuron đầu ra.

A $\Theta(h_1 + h_2 + h_3 - 1)$

B $\Theta(h_1 + h_2 + h_3 - 2)$

C $\Theta(h_1 + h_2 + h_3 - 3)$

D $\Theta(3 - h_1 - h_2 - h_3)$

PHỎNG THEO USA-NA-AIO 2026 · Vòng 1 · Part 7.4

LUYỆN ĐỀ 2

Mạng không có bias. $z_1 = w_A x_1 + w_B x_2$, $a_1 = g(z_1)$. Từ a_1 tách ra hai nhánh: $z_2 = w_C a_1$, $a_2 = g(z_2)$ và $z_3 = w_D a_1$, $a_3 = g(z_3)$. Cuối cùng $z_4 = w_E a_2 + w_F a_3$, $\hat{y} = g(z_4)$. Đặt $\delta = \partial \ell / \partial z_4$. Biểu thức nào là $\partial \ell / \partial a_1$?

A $\delta \left(w_E g'(z_2) w_C + w_F g'(z_3) w_D \right)$

B $\delta \, w_E g'(z_2) w_C$

C $\delta \left(w_E g'(z_2) w_C \right) \left(w_F g'(z_3) w_D \right)$

D $\delta \left(w_E w_C + w_F w_D \right)$

PHỎNG THEO CMU 10-601 · Exam 2 Practice · Q4.4

LUYỆN ĐỀ 3

Mạng một đầu vào, hai neuron ẩn sigmoid, đầu ra linear: $\hat{y} = v_1\sigma(w_1 x + c_1) + v_2\sigma(w_2 x + c_2) + d$. Khởi tạo hai neuron giống hệt nhau, tức $w_1 = w_2$, $c_1 = c_2$, $v_1 = v_2$, rồi chạy batch gradient descent với MSE. Chuyện gì xảy ra?

A Trọng số đứng yên, vì gradient bằng 0 ngay từ đầu

B Hai neuron mãi giống nhau, mạng không hơn một neuron

C Hai neuron tự tách nhau sau vài bước, nhờ nhiễu làm tròn

D Neuron thứ hai chết dần, chỉ neuron thứ nhất còn học

PHỎNG THEO MIT 6.036 · Final, Spring 2022 · Q4(h)

Chốt cơ chế

Hidden tạo feature · activation tạo phi tuyến.

Backward nhân độ nhạy, cộng các nhánh.

Làm 10 câu cuối bài, chạy notebook và kiểm lại bằng tay.